BÀI THỰC HÀNH MÔN HỌC MÁY – PRACTICE EXERCISE 3

# Phân cụm khách hàng (Customer Clustering)

Báo cáo thực hành chi tiết: Thiết kế chương trình, chuẩn hóa đặc trưng, đối chiếu bốn thuật toán phân cụm và phân tích chân dung phân khúc

Bài thực hành xây dựng một chương trình Python hoàn chỉnh cho bài toán học không giám sát **phân cụm khách hàng bán lẻ (Retail Customer Segmentation)** trên bộ dữ liệu thực tế `marketing_campaign.csv`. Trọng tâm của bài lab là kiểm tra và làm sạch dữ liệu có kiểm soát, chia tách tập Train/Holdout Test trước khi điền khuyết và chuẩn hóa để tránh rò rỉ dữ liệu, khảo sát bốn hướng thuật toán phổ biến (K-Means, Hierarchical Clustering, DBSCAN, K-Medoids), đánh giá định lượng qua các chỉ số nội tại và rút ra hồ sơ khách hàng phục vụ gợi ý tiếp thị ở quy mô học tập.

> **Ghi chú phương pháp & phạm vi bài lab:** Mọi bảng biểu, thông số và nhận xét trong báo cáo này được đối chiếu trực tiếp từ mã nguồn thực thi ( `main.py` , các module trong `src/` ) và tệp kết quả xuất ra tại `results/metrics/` cùng biểu đồ trong `results/figures/` . Các nhận định đều bám sát mạch: *Đầu vào → Lý do thiết kế → Công thức / Biến đổi → Kết quả thực tế → Giới hạn chứng cứ*, giữ đúng phạm vi một bài lab học tập chứ không phóng đại thành hệ thống vận hành doanh nghiệp.

Dữ liệu thô: 2.240 dòng x 29 cột | Sau lọc nhiễu: 2.236 bản ghi | Phân chia 80/20: Train 1.788 | Test 448 | Đặc trưng gom cụm: Age, Income, Total_Spending | Chuẩn hóa chọn: StandardScaler | Mô hình đại diện: K-Means (K = 2) | Train Silhouette: 0,4119 | Test Holdout: 0,4453

---

## 1. Đặt vấn đề, điều chỉnh dữ liệu từ đề gốc và tiêu chí đánh giá

### 1.1. Yêu cầu đề bài và thực tế bộ dữ liệu

Đề bài gốc trong `docs/Practice Exercise 3.pdf` mô tả bài toán phân cụm khách hàng cửa hàng bán lẻ từ 4 thông tin: *tuổi (age), giới tính (gender), thu nhập hàng năm (annual income)* và *điểm chi tiêu (spending score)*, yêu cầu thực hiện 4 bước: (1) Khám phá dữ liệu, (2) Chuẩn hóa đặc trưng, (3) Chọn và thử nghiệm tham số thuật toán phân cụm, (4) Đánh giá bằng chỉ số như Silhouette kết hợp biểu đồ Scatter/Dendrogram.

Khi kiểm tra tệp dữ liệu được cung cấp tại `data/raw/marketing_campaign.csv` (định dạng TSV phân tách tab, **2.240 dòng x 29 cột**), tập dữ liệu này thực chất không có sẵn 3 cột mang tên `Age` , `Gender` và `Spending Score (1-100)` . Thay vì tự bịa thêm dữ liệu giả hoặc bỏ qua sự chênh lệch này, bài lab xử lý minh bạch như sau:

- **Biến `Age`:** Quy đổi từ cột `Year_Birth` bằng công thức `Age = 2014 - Year_Birth`. Mốc `REFERENCE_YEAR = 2014` được chọn vì toàn bộ cột ngày gia nhập `Dt_Customer` trong tập dữ liệu nằm trong khoảng năm `2012–2014`.
- **Biến `Total_Spending`:** Cộng tổng 6 khoản chi tiêu sản phẩm có sẵn gồm `MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts` và `MntGoldProds`.
- **Biến nhân khẩu học:** Không tự tạo giả cột `Gender`. Hai cột phân loại thật là `Education` và `Marital_Status` chỉ giữ lại ở bước mô tả chân dung cụm sau khi phân nhóm, hoàn toàn không đưa vào tính khoảng cách để tránh làm nhiễu cấu trúc Euclidean 3 chiều.
- **Biến định danh và nhãn ngoài phạm vi:** Cột `ID` chỉ dùng làm khóa theo dõi bản ghi và loại khỏi ma trận đặc trưng; cột `Response` không dùng làm nhãn huấn luyện vì đây là bài toán học không giám sát.

### 1.2. Tiêu chí đánh giá phân cụm trong bài lab

Vì phân cụm không có nhãn đúng (ground truth) để tính Accuracy như bài toán phân loại, chất lượng gom nhóm được đánh giá tổng hợp theo 4 tiêu chí nội tại và độ cân đối hình học:

- **Silhouette Coefficient (chính):** Đo mức độ gắn kết trong cùng cụm so với khoảng cách tới cụm láng giềng gần nhất, nằm trong $[-1, 1]$ (càng gần $1$ càng rõ nét; ghi nhận thêm tỷ lệ điểm bị giá trị âm nằm ở vùng chồng lấn biên).
- **Davies–Bouldin Index (DBI):** Tỷ lệ độ phân tán nội cụm trên khoảng cách giữa hai tâm cụm; giá trị càng *thấp* chứng tỏ các cụm càng gọn và tách biệt.
- **Calinski–Harabasz Score (CHI):** Tỷ số phương sai giữa các cụm và phương sai trong cụm; giá trị càng *cao* càng tốt.
- **Dunn Index & Độ cân bằng cụm (Cluster Balance Ratio):** Kết hợp kiểm tra tỷ lệ $\min(\text{kích thước cụm}) / \max(\text{kích thước cụm})$ để tránh chọn nhầm những phân hoạch chỉ tách vài điểm ngoại lai ra làm một cụm riêng.

> **Vì sao bài lab không ấn định trước số cụm $K = 5$?**
>
> Nhiều ví dụ trực tuyến cho tập *Mall Customers* thường ra 5 cụm đặc trưng giữa thu nhập và điểm chi tiêu. Tuy nhiên, đề `Practice Exercise 3.pdf` không hề quy định trước số cụm phải là 5, và bộ dữ liệu thực tế ở đây là `marketing_campaign.csv` với phân phối khác hẳn. Vì vậy bài lab quét khách quan trên dải $K \in [2, 10]$ và chỉ chốt số cụm dựa trên kết quả đo đạc thực nghiệm.

## 2. Thiết kế chương trình và luồng xử lý dữ liệu

Mã nguồn được tổ chức thành các module tách biệt theo 4 bước của đề bài, kết nối qua điểm vào chính `main.py` cho luồng huấn luyện – đánh giá và lớp `CustomerSegmentPredictor` trong `src/deployment/predictor.py` cho luồng suy luận khách hàng mới.

### 2.1. Luồng huấn luyện và đánh giá chính ( `main.py` )

Đọc CSV & Kiểm định schema → Lọc 4 bản ghi lỗi miền → Chia 80% Train / 20% Test →
Điền trung vị Train & EDA → So sánh & Fit Scaler trên Train → Quét 4 thuật toán (K=2..10) →
Đánh giá Holdout & Profiling → Lưu .pkl & Thử nghiệm CLI

Toàn bộ quá trình huấn luyện được điều phối tuần tự từ `main.py` theo nguyên tắc tách biệt Train/Test ngay sau bước lọc lỗi miền (trước khi tính trung vị điền khuyết và trước khi fit bộ chuẩn hóa) để chặn rò rỉ dữ liệu. Sau khi quét tham số 4 thuật toán trên tập Train và kiểm chứng lại mô hình tốt nhất trên tập Holdout Test 20%, hệ thống xuất dữ liệu đã xử lý vào `data/processed/` , lưu bảng chỉ số – biểu đồ – mô hình đóng gói vào `results/` và gọi thử nghiệm suy luận ngay cuối pipeline.

### 2.2. Luồng suy luận dự đoán phân khúc khách hàng mới (Inference Pipeline)

Nạp bundle .pkl → Tiếp nhận & suy diễn cột → Kiểm tra số hữu hạn & miền → Điền trung vị & Scaler Train →
Predict cụm & Tra Persona

1. **Nạp artifact đã đóng băng:** Lớp `CustomerSegmentPredictor` đọc tệp `results/models/final_kmeans_pipeline.pkl` chứa `scaler` ( `FeatureScaler` đã fit trên Train), `model` ( `KMeans` ), và `metadata` (gồm `feature_medians` của Train, bảng `personas` , `reference_year` và `distance_metric` ).
2. **Tiếp nhận và chuẩn hóa cột đầu vào:** Ở chế độ đơn lẻ ( `predict_single_customer` ), hàm nhận trực tiếp bộ 3 giá trị `(Age, Income, Total_Spending)` ; ở chế độ hàng loạt ( `predict_batch` ), nếu dữ liệu có `Year_Birth` thì tự tính `Age = reference_year - Year_Birth` và nếu có đủ 6 cột `Mnt*` thì tính tổng thành `Total_Spending` .
3. **Kiểm định kiểu số và miền giá trị:** Các hàm `_numeric_column` và `_validate_features` từ chối giá trị không phải số, vô cùng ( `inf` / `-inf` ), `Age` nằm ngoài khoảng `(0, 120]` hoặc giá trị âm ở `Income` , `Total_Spending` và các cột `Mnt*` .
4. **Điền khuyết và co giãn theo tham số đóng băng:** Ở chế độ batch, các ô khuyết trong 3 đặc trưng cốt lõi được điền bằng đúng `feature_medians` của tập Train (chế độ đơn lẻ bắt buộc đủ 3 giá trị hợp lệ); sau đó toàn bộ ma trận đặc trưng được co giãn bằng `scaler.transform()` mà tuyệt đối không fit lại trên dữ liệu mới.
5. **Gán nhãn cụm và ánh xạ khuyến nghị:** Gọi `model.predict()` trên không gian đã chuẩn hóa để thu về `Predicted_Cluster` , tính khoảng cách tới tâm cụm (Manhattan `cityblock` hoặc Euclidean trong không gian chuẩn hóa đối với dự đoán đơn lẻ), và tra `Persona_Name` cùng `Marketing_Recommendation` tương ứng từ `metadata["personas"]` .

### 2.3. Bảng phân bổ trách nhiệm các module

Cấu trúc các module mã nguồn và tệp đầu ra chính của bài lab

| Nhóm module | Tệp nguồn | Nhiệm vụ chính | Đầu ra tạo ra |
|---|---|---|---|
| **Cấu hình & Log** | `src/config.py` , `src/utils/logger.py` | Định nghĩa đường dẫn dữ liệu/kết quả, cố định `RANDOM_SEED=42` , bảng màu, ghi log thực thi | `results/metrics/pipeline_execution.log` |

| Nhóm module | Tệp nguồn | Nhiệm vụ chính | Đầu ra tạo ra |
|---|---|---|---|
| **Dữ liệu & Tiền xử lý** | `src/data/loader.py`, `src/data/preprocessing.py` | Nạp tệp TSV, kiểm tra schema, suy diễn `Age` và `Total_Spending`, lọc dị biệt, chia 80/20, điền trung vị, audit IQR | `train.csv`, `test.csv`, `raw_data_audit.csv`, `train_iqr_audit.csv` |
| **Chuẩn hóa đặc trưng** | `src/features/scaling.py` | Cài đặt `FeatureScaler` (Standard, MinMax, Robust), kiểm tra hậu điều kiện Train/Test, đối chiếu tác động lên K-Means | `scaler_params.csv`, `scaler_comparison.csv`, `scaler_verification.csv`, `scaler_clustering_comparison.csv` |
| **Mô hình phân cụm** | `src/models/kmeans.py`, `hierarchical.py`, `dbscan.py`, `kmedoids.py`, `persistence.py` | Huấn luyện, quét tham số 4 thuật toán và đóng gói pipeline `final_kmeans_pipeline.pkl` | 8 bảng CSV quét tham số & tệp mô hình `.pkl` |
| **Đánh giá & Trực quan** | `src/evaluation/metrics.py`, `visualization.py`, `profiling.py` | Tính Silhouette, DBI, CHI, Dunn; vẽ 10 biểu đồ PNG; lập bảng thống kê cụm và sinh mô tả phân khúc động | 10 ảnh PNG, `model_comparison.csv`, `holdout_test_evaluation.csv`, `cluster_profiles.csv` |
| **Suy luận thử nghiệm** | `src/deployment/predictor.py` | Nạp bundle `.pkl`, tự điền khuyết theo trung vị Train nếu thiếu `Income`, gán cụm 1 khách hàng hoặc hàng loạt qua CLI | `test_predictions.csv` và kết quả dự đoán trên terminal |

## 3. Nạp, kiểm toán dữ liệu gốc và phân tách 80/20

### 3.1. Kiểm toán dữ liệu thô trước khi làm sạch

Tệp `data/raw/marketing_campaign.csv` chứa đúng **2.240 dòng x 29 cột** với mã định danh `ID` duy nhất hoàn toàn (0 bản ghi trùng `ID`, 0 ô thiếu `ID`). Khi rà soát theo từng cột và kiểm tra ngoại lai bằng khoảng tứ phân vị IQR trên tập thô (`results/metrics/raw_data_audit.csv`), ghi nhận các điểm bất thường sau:

Kiểm toán giá trị khuyết và ngoại lai IQR trên 2.240 bản ghi gốc (`raw_data_audit.csv`)

| Đặc trưng | Null Count | Tỷ lệ Null | Q1 | Q3 | IQR | Ngưỡng dưới | Ngưỡng trên | Số Outlier IQR | Tỷ lệ Outlier |
|---|---|---|---|---|---|---|---|---|---|
| `Age` | 0 | 0,00% | 37,0 | 55,0 | 18,0 | 10,00 | 82,00 | 3 | 0,13% |
| `Income` | 24 | 1,07% | 35.303,0 | 68.522,0 | 33.219,0 | -14.525,50 | 118.350,50 | 8 | 0,36% |
| `Total_Spending` | 0 | 0,00% | 68,75 | 1.045,5 | 976,75 | -1.396,38 | 2.510,62 | 3 | 0,13% |

- **Các bản ghi dị biệt cực đoan (lỗi miền giá trị):** 3 khách hàng có năm sinh nhỏ hơn `1920` gồm `ID 7829` (`1900`), `ID 11004` (`1893`), `ID 1150` (`1899`) tương ứng 114–121 tuổi; cùng 1 khách hàng có thu nhập vượt trội phi thực tế là `ID 9432` (`Income = 666.666,0`, cao gấp hơn 4 lần mức cao nhất kế tiếp là `162.397,0`). Tổng cộng có **4 bản ghi** cần lọc bỏ trước khi phân cụm.
- **Các nhãn hiếm ở cột `Marital_Status`:** Dữ liệu gốc có `Married (864)`, `Together (580)`, `Single (480)`, `Divorced (232)`, `Widow (77)`, và một số nhãn lạ gồm `Alone (3)`, `Absurd (2)`, `YOLO (2)`. Các nhãn này được giữ nguyên vì cột này chỉ dùng đọc thống kê mô tả cụm sau cùng.

### 3.2. Phân tách tập Train / Holdout Test và chống rò rỉ dữ liệu

Sau khi loại 4 bản ghi lỗi miền (còn **2.236 mẫu**), dữ liệu được chia ngẫu nhiên cố định theo tỷ lệ **80% Train ( `1.788` mẫu) / 20% Holdout Test ( `448` mẫu)** với `RANDOM_SEED = 42`. Bước tính trung vị `Income = 51.650,50` chỉ thực hiện trên `1.788` mẫu của Train rồi dùng giá trị này điền khuyết cho **18 ô thiếu trong Train** và **6 ô thiếu trong Test**.

> **Vì sao giữ lại các bản ghi trùng trên 3 đặc trưng số và các điểm IQR hợp lệ ở tập Train?**
>
> Sau khi làm sạch, tập Train còn `136` bản ghi trùng nhau hoàn toàn trên 3 chiều `[Age, Income, Total_Spending]` nhưng khác mã `ID`; đây là các khách hàng thật có cùng tuổi, mức thu nhập và tổng chi tiêu, không phải lỗi trùng lặp dòng ( `ID` trùng bằng 0). Ngoài ra, phân tích IQR trên tập Train ( `train_iqr_audit.csv` ) chỉ còn `7` mẫu `Income > 117.654,75` (tối đa `162.397,0`, chiếm `0,39%`) và `1` mẫu `Total_Spending > 2.505,25` ( `2.524,0`, chiếm `0,06%`). Đây đều là khách hàng thu nhập/chi tiêu cao hoàn toàn hợp lý nên được giữ nguyên để mô hình phản ánh đúng tệp khách hàng giàu.

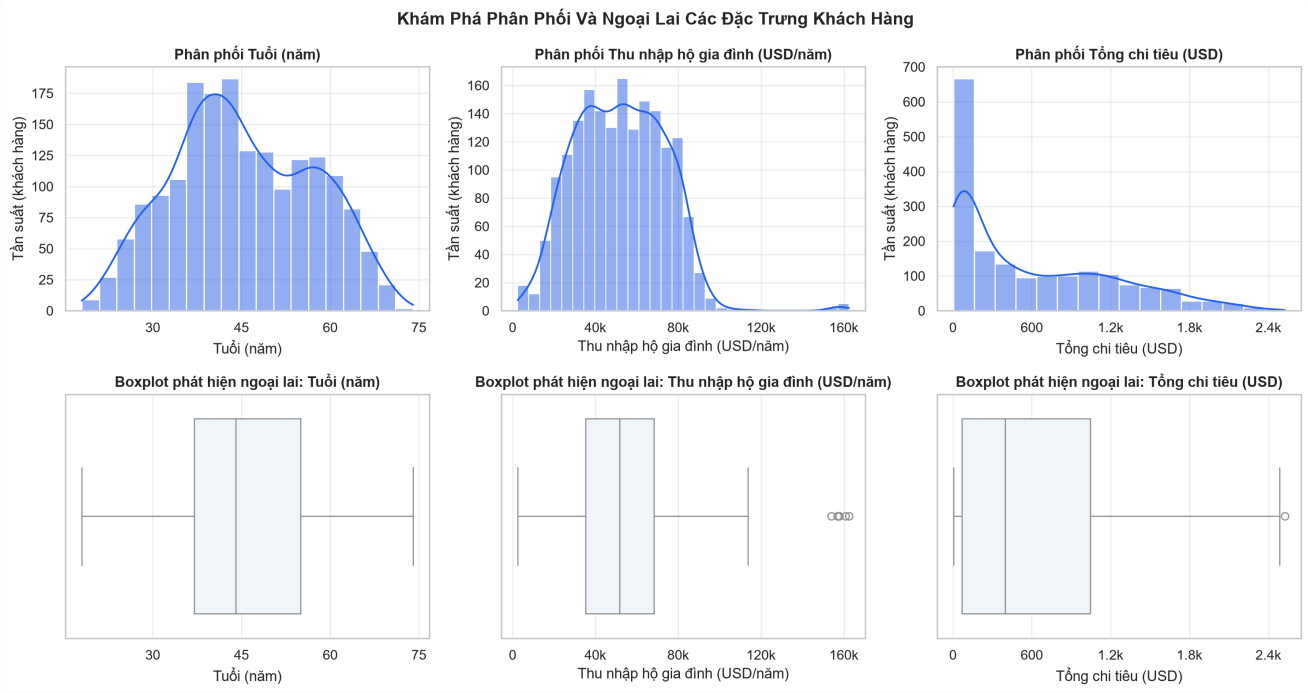
## 4. Khám phá phân phối và tương quan đặc trưng (EDA)

Trên tập Train ( `1.788` mẫu đã làm sạch), các đặc trưng số có hình thái phân phối khác biệt khá rõ:

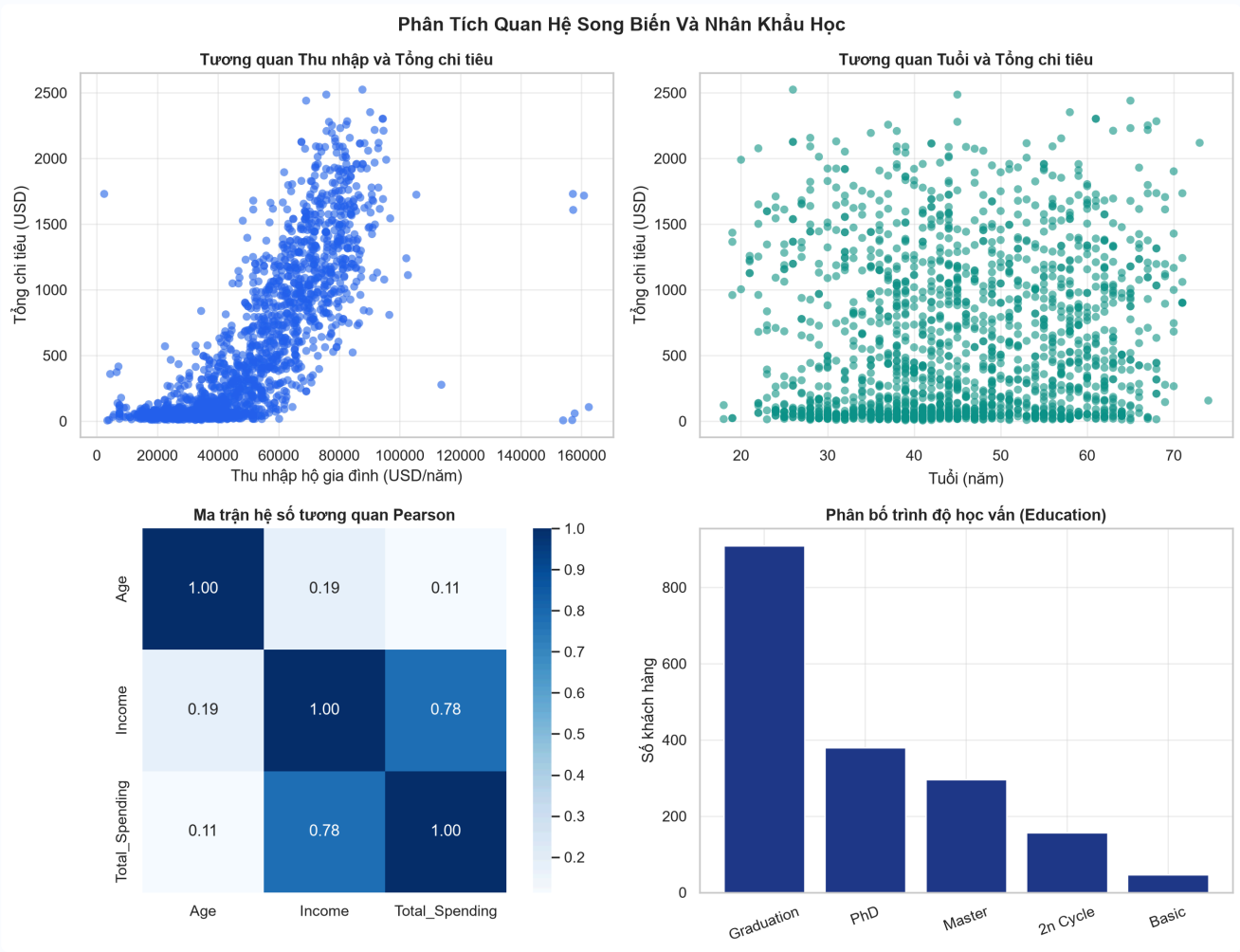
Thống kê mô tả và kiểm toán IQR trên tập Train (1.788 mẫu)

| Đặc trưng | Min | Q1 | Median | Mean | Q3 | Max | Std (ddof=0) | Skewness | Outlier IQR |
|---|---|---|---|---|---|---|---|---|---|
| `Age` | 18,0 | 37,0 | 44,0 | 45,23 | 55,0 | 74,0 | 11,85 | 0,095 | 0 (0,00%) |
| `Income` | 2.447,0 | 35.196,0 | 51.650,5 | 51.975,56 | 68.179,5 | 162.397,0 | 21.560,98 | 0,413 | 7 (0,39%) |
| `Total_Spending` | 5,0 | 69,0 | 397,5 | 605,60 | 1.043,5 | 2.524,0 | 598,65 | 0,857 | 1 (0,06%) |

Biến `Age` phân phối khá đều quanh trung tâm 44–45 tuổi. Biến `Income` có trung bình xấp xỉ trung vị (~52 nghìn USD) với đuôi phải vừa phải. Riêng `Total_Spending` lệch phải mạnh ( `skew = 0,8565`, trung vị chỉ `397,5` nhưng trung bình lên tới `605,6` và đỉnh ở `2.524`), phản ánh phần đông khách hàng chi tiêu ít và một nhóm khách hàng có thu nhập cao chi tiêu mạnh tạo thành cụm kéo dài về phía trên bên phải biểu đồ phân tán.




Hình 1: Phân phối Histogram/KDE và Boxplot phát hiện ngoại lai cho Age, Income, Total_Spending (Train)




Hình 2: Quan hệ song biến (Income vs Total_Spending, Age vs Total_Spending), ma trận tương quan và học vấn

## 5. Chuẩn hóa đặc trưng và đối chiếu tác động (Feature Scaling)

### 5.1. Lý do bắt buộc phải chuẩn hóa và công thức 3 bộ biến đổi

Ở thang đo gốc, độ lệch chuẩn của `Income` ( `21.560,98` ) lớn gấp gần **1.820 lần** so với `Age` ( `11,85` ). Nếu tính khoảng cách Euclidean bình phương $d^2(x, y) = \sum_{j=1}^{3}(x_j - y_j)^2$ mà không chuẩn hóa, chênh lệch thu nhập vài nghìn USD sẽ lấn át hoàn toàn chênh lệch tuổi tác hay chi tiêu.

$$\text{StandardScaler:}\quad z_j = \frac{x_j - \mu_{\text{train},j}}{\sigma_{\text{train},j}}$$

$$\text{MinMaxScaler:}\quad x'_j = \frac{x_j - x_{\min,\text{train},j}}{x_{\max,\text{train},j} - x_{\min,\text{train},j}}$$

$$\text{RobustScaler:}\quad \tilde{x}_j = \frac{x_j - \text{Median}_{\text{train},j}}{\text{IQR}_{\text{train},j}}$$

Các tham số $\mu, \sigma, x_{\min}, x_{\max}, \text{Median}, \text{IQR}$ được học **duy nhất trên 1.788 mẫu Train** (ví dụ với `StandardScaler` trong `scaler_params.csv`: `Age` $\mu = 45{,}2299, \sigma = 11{,}8506$; `Income` $\mu = 51.975{,}5587, \sigma = 21.560{,}9848$; `Total_Spending` $\mu = 605{,}5990, \sigma = 598{,}6500$).

### 5.2. So sánh phân phối thang đo và kiểm chứng hậu điều kiện Train/Test

Thống kê phân phối tập Train qua 4 trạng thái thang đo (`scaler_comparison.csv`)

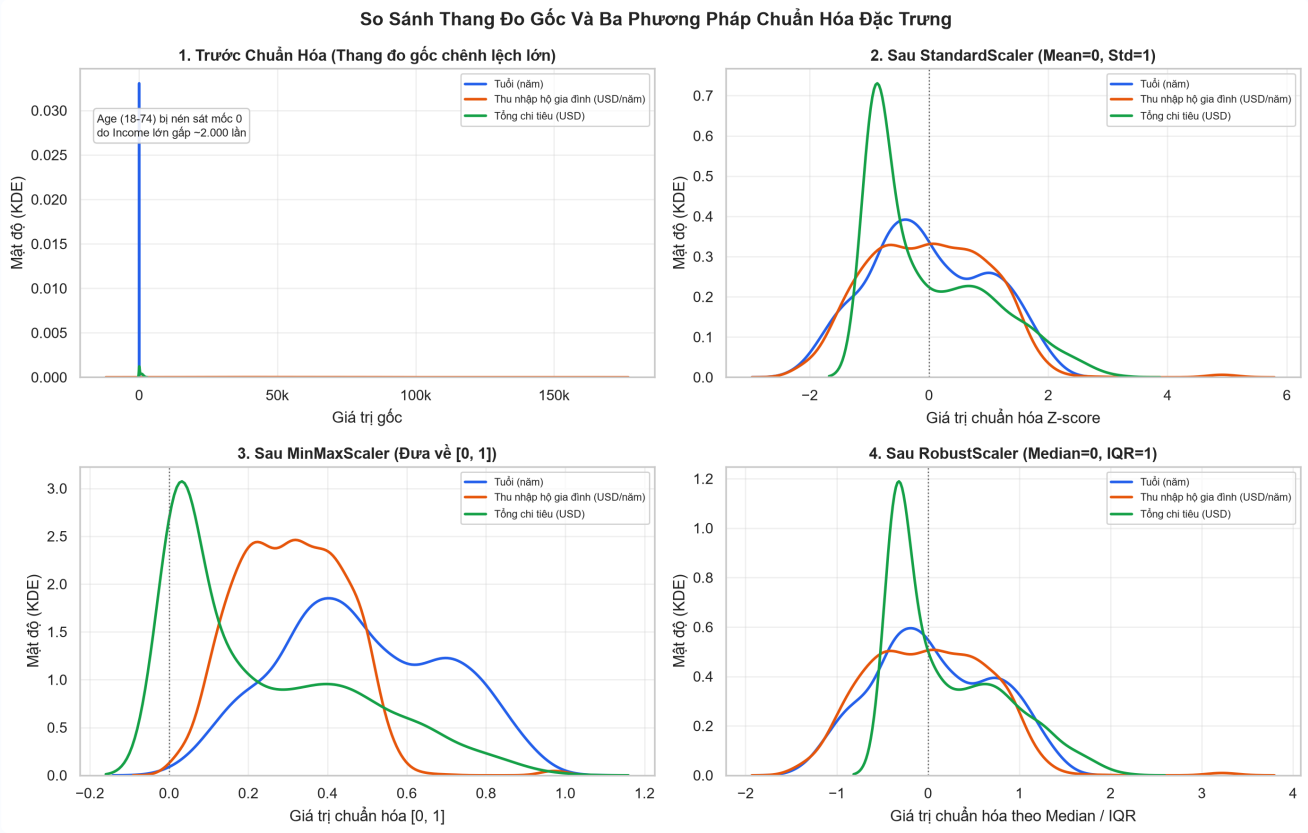
| Phương pháp | Đặc trưng | Min | Max | Mean | Std | Median | IQR | Skew |
|---|---|---|---|---|---|---|---|---|
| Gốc (Raw) | Age | 18,0000 | 74,0000 | 45,2299 | 11,8506 | 44,0000 | 18,0000 | 0,0950 |
| Gốc (Raw) | Income | 2.447,0000 | 162.397,0000 | 51.975,5587 | 21.560,9848 | 51.650,5000 | 32.983,5000 | 0,4129 |
| Gốc (Raw) | Total_Spending | 5,0000 | 2.524,0000 | 605,5990 | 598,6500 | 397,5000 | 974,5000 | 0,8565 |
| StandardScaler | Age | -2,2978 | 2,4277 | 0,0000 | 1,0000 | -0,1038 | 1,5189 | 0,0950 |
| StandardScaler | Income | -2,2971 | 5,1214 | 0,0000 | 1,0000 | -0,0151 | 1,5298 | 0,4129 |
| StandardScaler | Total_Spending | -1,0033 | 3,2045 | 0,0000 | 1,0000 | -0,3476 | 1,6278 | 0,8565 |
| MinMaxScaler | Age | 0,0000 | 1,0000 | 0,4862 | 0,2116 | 0,4643 | 0,3214 | 0,0950 |
| MinMaxScaler | Income | 0,0000 | 1,0000 | 0,3097 | 0,1348 | 0,3076 | 0,2062 | 0,4129 |
| MinMaxScaler | Total_Spending | 0,0000 | 1,0000 | 0,2384 | 0,2377 | 0,1558 | 0,3869 | 0,8565 |
| RobustScaler | Age | -1,4444 | 1,6667 | 0,0683 | 0,6584 | 0,0000 | 1,0000 | 0,0950 |
| RobustScaler | Income | -1,4918 | 3,3576 | 0,0099 | 0,6537 | 0,0000 | 1,0000 | 0,4129 |
| RobustScaler | Total_Spending | -0,4028 | 2,1821 | 0,2135 | 0,6143 | 0,0000 | 1,0000 | 0,8565 |

Khi áp tham số Train sang **448 mẫu Holdout Test** (`scaler_verification.csv`): `StandardScaler` giữ ổn định (`Mean` từ `-0,0539` đến `+0,0032`; `Std` từ `0,9339` đến `1,0254`). Trong khi đó, `MinMaxScaler` khiến **3 mẫu Test vượt ra ngoài đoạn** `[0, 1]`: `1` mẫu có `Income = -0,0045` (dưới cận min Train `2.447,0`) và `2` mẫu có `Total_Spending` lên tới `1,0004` (vượt cận max Train `2.524,0`).

Đối chiếu thực nghiệm 3 phương pháp chuẩn hóa khi chạy K-Means (`scaler_clustering_comparison.csv`)

| Scaler | K tối ưu | Silhouette Train | DBI Train | CHI Train | Inertia Train | Cân bằng cụm | Silhouette (K=5) | DBI (K=5) | Silhouette Test |
|---|---|---|---|---|---|---|---|---|---|
| **standard (Chọn)** | **2** | **0,4119** | **0,9892** | **1.495,38** | **2.919,54** | **0,7444** | **0,3527** | **0,9416** | **0,4453** |

| Scaler | K tối ưu | Silhouette Train | DBI Train | CHI Train | Inertia Train | Cân bằng cụm | Silhouette (K=5) | DBI (K=5) | Silhouette Test |
|---|---|---|---|---|---|---|---|---|---|
| minmax | 2 | 0,4210 | 0,9963 | 1.463,04 | 117,39 | 0,6602 | 0,3812 | 0,8994 | 0,4561 |
| robust | 2 | 0,4026 | 1,0127 | 1.422,45 | 1.232,32 | 0,7598 | 0,3528 | 0,9412 | 0,4368 |




Hình 3: Đối chiếu thang đo trước và sau khi chuẩn hóa bằng StandardScaler trên tập Train

> **Vì sao chọn `StandardScaler` thay vì `MinMaxScaler` dù Silhouette của `MinMaxScaler` cao hơn nhẹ (0,4210 vs 0,4119)?**
>
> Trong bảng phân phối trên tập Train, vì `Income` có một số khách hàng thu nhập cao tới `162.397` nên `MinMaxScaler` nén phần lớn `Income` về dải hẹp ( `Std = 0,1348` ), nhỏ hơn đáng kể so với độ trải của `Total_Spending` ( `Std = 0,2377` ). Việc thu hẹp trọng số tương đối của trục thu nhập giúp Silhouette tăng nhẹ lên `0,4210` , nhưng đánh đổi bằng chỉ số Davies–Bouldin xấu hơn ( `0,9963 > 0,9892` ), Calinski–Harabasz thấp hơn ( `1.463,04 < 1.495,38` ), hai cụm mất cân đối hơn ( `0,6602` vs `0,7444` ) và nhạy với điểm vượt biên `[0, 1]` ở tập mới. `StandardScaler` đảm bảo mọi trục đều có phương sai đơn vị $\sigma = 1$ nên cân bằng và ổn định hơn.

## 6. Nền tảng toán học của bốn thuật toán phân cụm

Tất cả thuật toán đều làm việc trên cùng ma trận đã chuẩn hóa bằng `StandardScaler` kích thước $1.788 \times 3$ với hạt giống ngẫu nhiên `RANDOM_SEED = 42` .

### 6.1. K-Means (Baseline chính)

K-Means chia tập dữ liệu $\{x_1, \ldots, x_N\} \subset \mathbb{R}^3$ thành $K$ cụm $C_1, \ldots, C_K$ sao cho tối thiểu hóa tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm $\mu_k$ (Inertia / WCSS):

$$\min_{\{C_k, \mu_k\}} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2, \qquad \mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Bài lab sử dụng khởi tạo `k-means++` (chọn các tâm ban đầu cách xa nhau theo phân phối xác suất tỷ lệ với $D(x)^2$) nhằm giảm nguy cơ kẹt cực tiểu cục bộ so với chọn ngẫu nhiên thuần túy.

### 6.2. Hierarchical Clustering (Agglomerative)

Phân cụm phân cấp đi từ dưới lên, ban đầu coi mỗi mẫu là một cụm đơn rồi gộp dần cặp cụm gần nhất theo tiêu chuẩn khoảng cách liên kết (linkage):

- **`ward`:** Gộp cặp cụm $(A, B)$ làm tăng tổng phương sai nội cụm ít nhất ($\Delta\text{SSE} = \frac{|A||B|}{|A|+|B|}\|\mu_A - \mu_B\|_2^2$), có xu hướng tạo cụm gọn và cân đối.
- **`complete`** **và** **`average`:** Đo lần lượt theo khoảng cách lớn nhất hoặc trung bình giữa mọi cặp điểm thuộc hai cụm. Chất lượng cây phân cấp so với ma trận khoảng cách gốc được đo bằng *Hệ số tương quan Cophenetic*.

### 6.3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN gom nhóm theo mật độ cục bộ dựa trên hai tham số bán kính lân cận $\varepsilon$ ( `eps` ) và số điểm tối thiểu `min_samples` . Một điểm là *điểm lõi (core point)* nếu trong hình cầu bán kính $\varepsilon$ quanh nó có ít nhất `min_samples` điểm; các điểm không nối được tới điểm lõi nào bị gán nhãn nhiễu `-1` . Thuật toán không cần truyền trước $K$ và có thể tách vùng thưa thành nhiễu, nhưng nhạy cảm với mật độ không đồng đều.

### 6.4. K-Medoids (Tự cài đặt pha BUILD tất định & cập nhật nội cụm)

Trong `src/models/kmedoids.py` , thuật toán K-Medoids được tự cài đặt bằng NumPy/SciPy (không phụ thuộc thư viện ngoài), luôn chọn các **điểm khách hàng thật** làm tâm đại diện (medoid $m_k \in X$) để giảm ảnh hưởng của giá trị ngoại lai:

$$\min_{\{m_1, \ldots, m_K\} \subset X} \sum_{i=1}^{N} \min_{1 \le k \le K} d(x_i, m_k)$$

Pha khởi tạo `BUILD` chọn medoid đầu tiên là điểm có tổng khoảng cách tới toàn bộ tập nhỏ nhất, các medoid kế tiếp chọn ứng viên giảm tổng chi phí nhiều nhất; sau đó lặp lại bước gán cụm Voronoi và cập nhật medoid mới trong từng cụm cho đến khi hội tụ (hỗ trợ cả khoảng cách `euclidean` lẫn `manhattan` ).

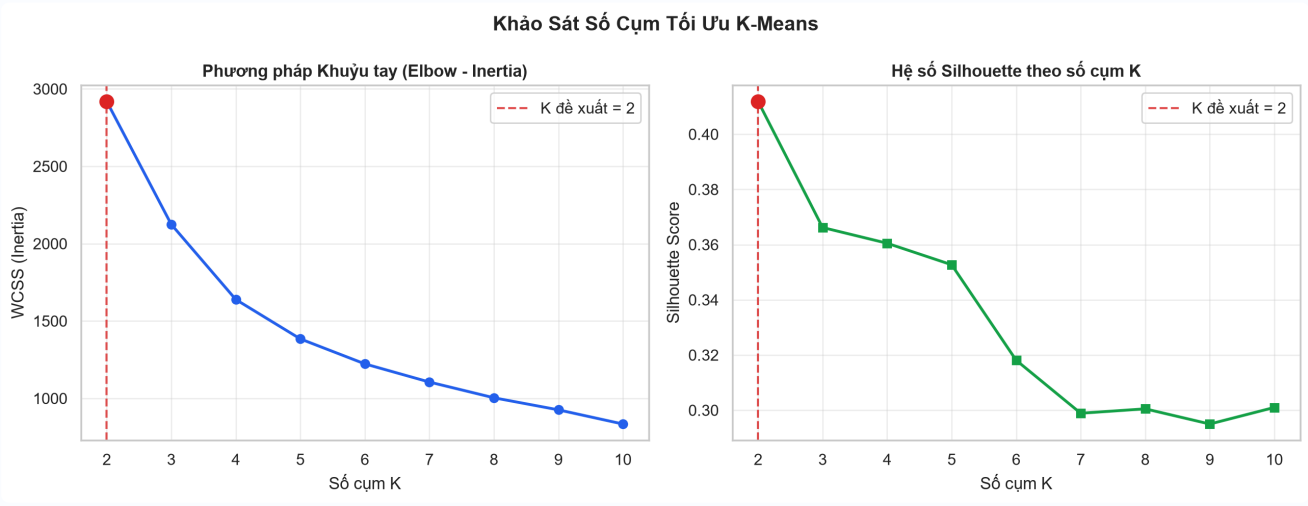
# 7. Thực nghiệm quét tham số trên từng thuật toán

## 7.1. K-Means: Quét $K = 2 \dots 10$ và thử nghiệm siêu tham số

Kết quả quét số cụm K cho K-Means ( `kmeans_k_search.csv` )

| Số cụm $K$ | Inertia | Silhouette Score | Davies–Bouldin | Calinski–Harabasz |
|---|---|---|---|---|
| **2** | **2.919,5358** | **0,4119** | **0,9892** | **1.495,38** |
| 3 | 2.124,7672 | 0,3662 | 0,9606 | 1.360,63 |
| 4 | 1.639,1406 | 0,3605 | 1,0051 | 1.351,38 |
| 5 | 1.384,5151 | 0,3527 | 0,9416 | 1.281,22 |
| 6 | 1.223,0876 | 0,3181 | 0,9570 | 1.206,64 |
| 7 | 1.105,5846 | 0,2989 | 1,0343 | 1.143,33 |
| 8 | 1.004,0989 | 0,3005 | 0,9696 | 1.104,15 |
| 9 | 925,9026 | 0,2950 | 0,9825 | 1.065,93 |
| 10 | 834,3847 | 0,3010 | 0,9494 | 1.072,47 |

Tại $K = 2$ (đạt Silhouette cao nhất `0,4119` và Calinski–Harabasz cao nhất `1.495,38` ), khi thử nghiệm 8 tổ hợp siêu tham số gồm `init ∈ {k-means++, random}` , `n_init ∈ {10, 20}` , `max_iter ∈ {100, 300}` ( `kmeans_hyperparams.csv` ), cả 8 cấu hình đều hội tụ cùng một nghiệm tối ưu ( `Inertia = 2.919,5358` , `Silhouette = 0,4119` , `DBI = 0,9892` ), riêng `k-means++` hội tụ nhanh hơn một nhịp (**8 vòng lặp** so với **9 vòng lặp** của `random` ).




Hình 4: Đường cong Elbow (Inertia) và Silhouette Score khi quét K từ 2 đến 10 của K-Means

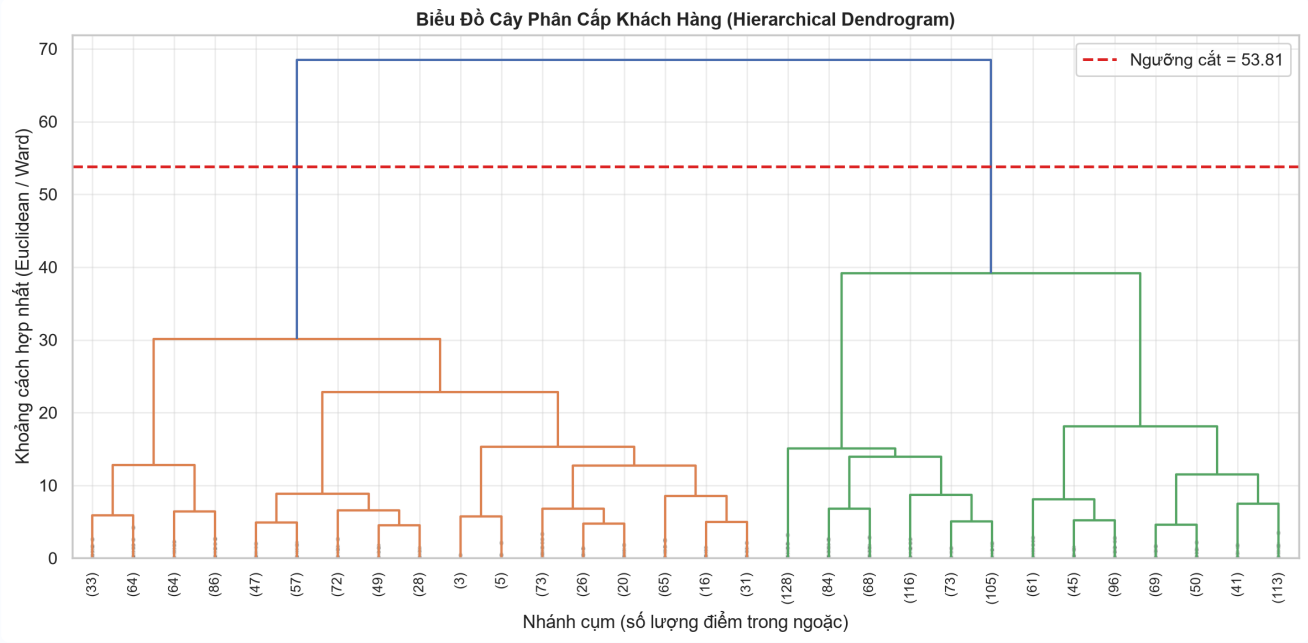
## 7.2. Hierarchical Clustering: So sánh Linkage và Quét $K$

Đối chiếu các phương pháp liên kết tại K = 2 ( `hierarchical_cophenetic.csv` )

| Linkage | Cophenetic Correlation | K=2 Silhouette | K=2 Balance Ratio | Kích thước cụm tại K=2 |
|---|---|---|---|---|

| Linkage | Cophenetic Correlation | K=2 Silhouette | K=2 Balance Ratio | Kích thước cụm tại K=2 |
|---|---|---|---|---|
| **ward** | **0,6137** | **0,3948** | **0,7045** | **{0: 1.049, 1: 739}** |
| complete | 0,5958 | 0,5701 | 0,0045 | {0: 1.780, 1: 8} |
| average | 0,6940 | 0,5701 | 0,0045 | {0: 1.780, 1: 8} |

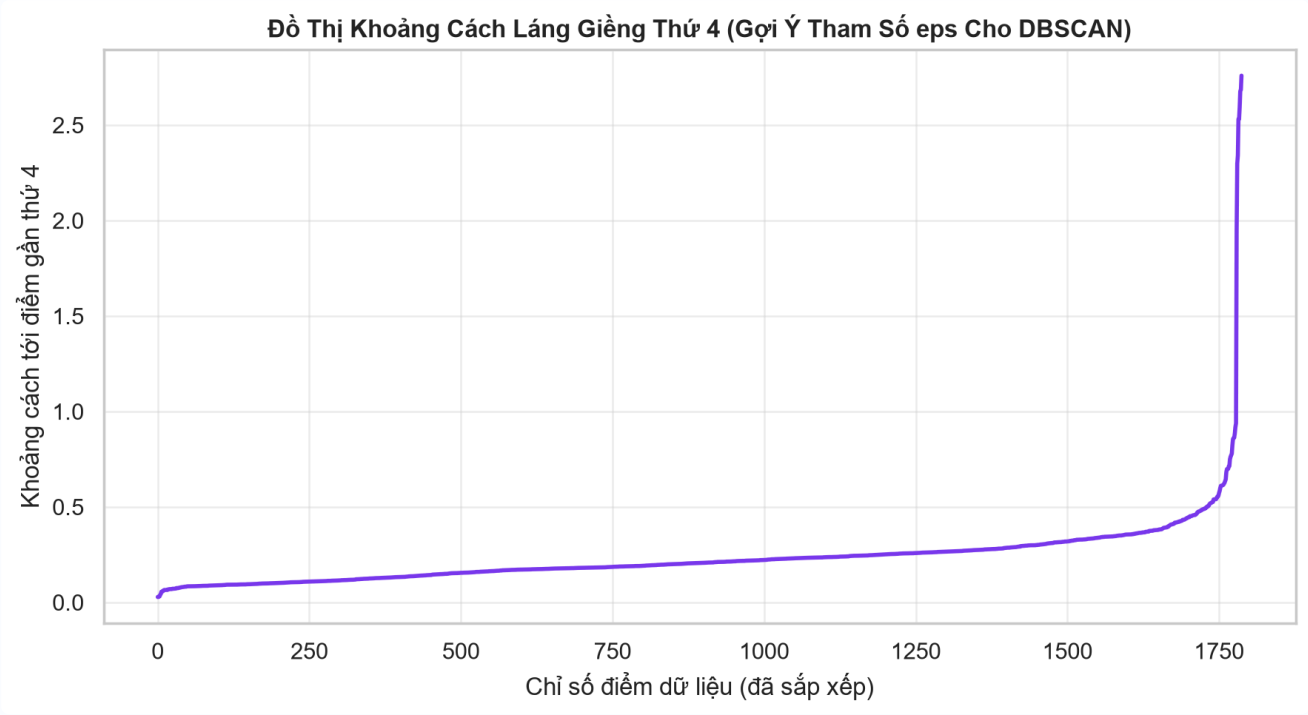
**Phát hiện đáng chú ý:** Dù `average` đạt Cophenetic cao nhất ( `0,6940` ) và cùng `complete` có Silhouette tại $K = 2$ tới `0,5701` , cả hai đều bị **hiệu ứng xích (chaining)**: tách đúng 8 khách hàng thu nhập/chi tiêu cực cao ở rìa thành một cụm tí hon và dồn toàn bộ `1.780` người còn lại vào cụm còn lại (độ cân bằng chỉ `0,0045` ). Do đó bài lab chọn **`ward`** làm đối chứng chính vì tạo phân tách có nghĩa ( `1.049` và `739` mẫu). Khi quét $K = 2 \dots 10$ với `ward` ( `hierarchical_k_search.csv` ), Silhouette cũng đạt đỉnh ở $K = 2$ **( `0,3948` )** rồi giảm dần ( `K=3: 0,3333` , `K=4: 0,3206` , `K=5: 0,3088` , `K=10: 0,2413` ).



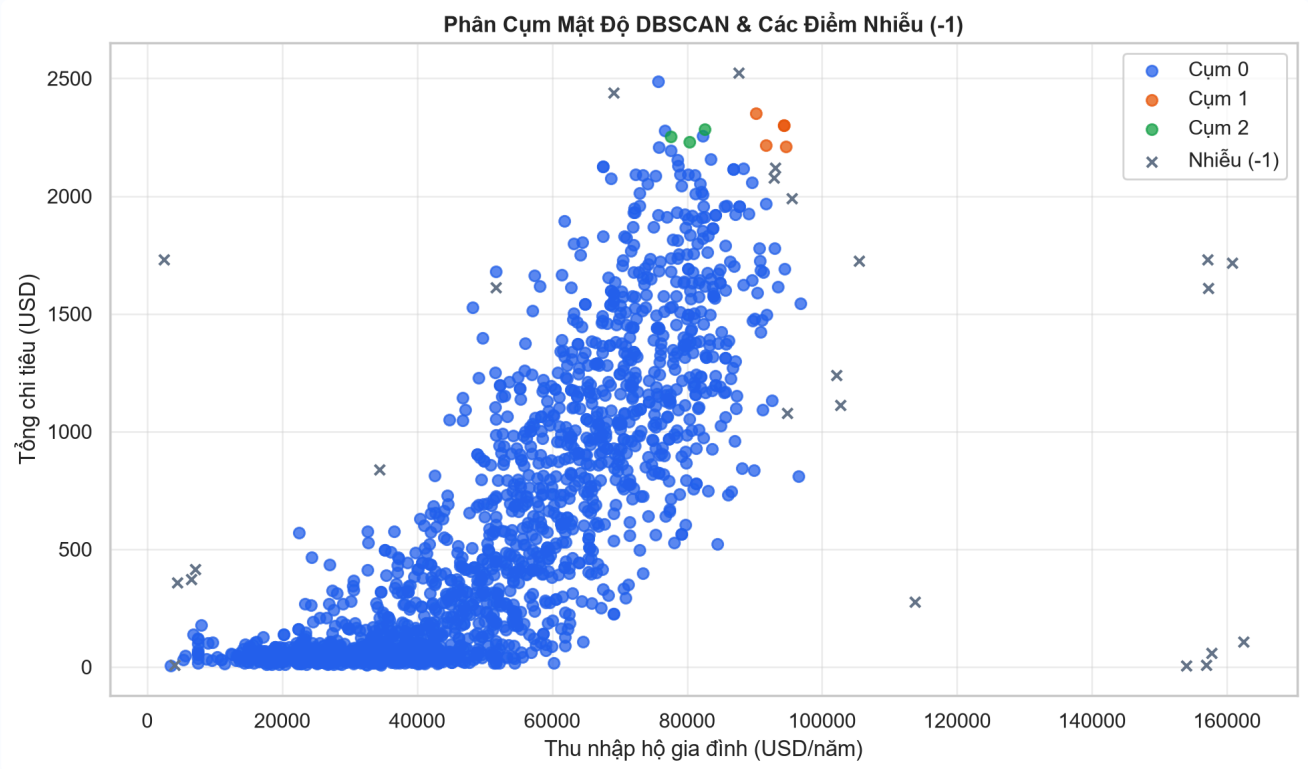

Hình 5: Dendrogram của Hierarchical Clustering (Ward linkage) kèm đường cắt ngang tại K = 2

### 7.3. DBSCAN: Đồ thị k-NN và lưới tìm kiếm tham số

Lưới tham số quét 24 tổ hợp với `eps ∈ {0.3, 0.4, 0.5, 0.6, 0.7, 0.8}` và `min_samples ∈ {4, 5, 8, 10}` ( `dbscan_grid_search.csv` ). Những cấu hình có `eps ≥ 0,6` (hoặc `eps=0,5, min_samples ≥ 5` và `eps=0,4, min_samples ∈ {5, 8}` ) gộp phần lõi thành **duy nhất 1 cụm** nên không đủ điều kiện tính Silhouette (code trả `NaN` thay vì gán giả giá trị `0` ). Cấu hình tốt nhất trong lưới là `eps = 0,5, min_samples = 4` đạt Silhouette (không tính nhiễu) `0,3596` với `3` cụm và `24` điểm nhiễu ( `1,34%` ); cấu hình có 2 cụm rõ hơn là `eps = 0,4, min_samples = 10` nhưng bỏ nhiễu tới `132` điểm ( `7,38%` , Silhouette `0,3233` ).




Hình 6: Đồ thị khoảng cách k-NN (k = 4, đã loại tự khoảng cách 0) hỗ trợ xác định điểm khuỷu tay cho eps




Hình 7: Phân cụm DBSCAN (eps = 0,5, min_samples = 4) tách 1 cụm lớn, 2 vi cụm rìa và 24 điểm nhiễu (-1)

### 7.4. K-Medoids: Quét $K$ và so sánh thước đo khoảng cách

Quét $K = 2 \dots 10$ với thước đo `euclidean` ( `kmedoids_k_search.csv` ) cũng hội tụ đỉnh Silhouette tại $K = 2$ **( `0,4080` )** ( `Total_Cost = 2.057,1048` ; so với `K=3: 0,3663` , `K=4: 0,2870` , `K=5: 0,3240` ). Đối chiếu hai thước đo tại $K = 2$ ( `kmedoids_metric_comparison.csv` ) cho thấy `euclidean` ( `Silhouette = 0,4080` , `DBI = 0,9918` , `CHI = 1.486,32` ) nhỉnh nhẹ hơn `manhattan` ( `Silhouette = 0,4033` , `DBI = 1,0028` , `CHI = 1.436,99` , `Total_Cost = 2.965,3186` ).

## 8. Đánh giá tổng hợp, kiểm chứng Holdout Test và trực quan hóa

### 8.1. Bảng đối chiếu 4 mô hình trên tập Train (1.788 mẫu)

So sánh tổng hợp 4 thuật toán phân cụm trên tập Train ( `model_comparison.csv` )

| Thuật toán | Số cụm K | Mẫu hợp lệ | Điểm nhiễu (%) | Silhouette Score | Silhouette âm (%) | Davies–Bouldin | Calinski–Harabasz | Dunn Index | Độ cân bằng cụm |
|---|---|---|---|---|---|---|---|---|---|
| **K-Means (K=2)** | **2** | **1.788** | **0 (0,00%)** | **0,4119** | **1,68%** | **0,9892** | **1.495,38** | **0,0161** | **0,7444** |
| K-Medoids (K=2) | 2 | 1.788 | 0 (0,00%) | 0,4080 | 2,80% | 0,9918 | 1.486,32 | 0,0106 | 0,8079 |
| Hierarchical Ward (K=2) | 2 | 1.788 | 0 (0,00%) | 0,3948 | 5,03% | 1,0195 | 1.387,73 | 0,0200 | 0,7045 |
| DBSCAN (eps=0.5, ms=4) | 3 | 1.764 | 24 (1,34%) | 0,3596 | 14,51% | 0,5214 | 19,06 | 0,0704 | 0,0017 |

Cả 3 mô hình dựa trên khoảng cách ( `K-Means` , `K-Medoids` , `Hierarchical Ward` ) cùng đạt đỉnh tối ưu tại $K = 2$. Trong đó, `K-Means (K=2)` được chọn làm mô hình cuối cùng vì đạt Silhouette cao nhất ( `0,4119` ), Calinski–Harabasz cao nhất ( `1.495,38` ), Davies–Bouldin tốt nhất trong nhóm phân hoạch cân đối ( `0,9892` ) và tỷ lệ điểm có Silhouette âm thấp nhất ( `1,68%` ). Mặc dù `DBSCAN` có DBI ( `0,5214` ) và Dunn Index ( `0,0704` ) nhìn bề ngoài tốt hơn, đó chỉ là hệ quả hình học khi tách 2 vi cụm siêu nhỏ (5 điểm và 3 điểm) ở rìa khỏi 1 khối lớn 1.756 điểm (cân bằng chỉ `0,0017` ), không có giá trị phân khúc khách hàng.

### 8.2. Kiểm chứng độ ổn định phân bố cụm trên tập Holdout Test (448 mẫu)

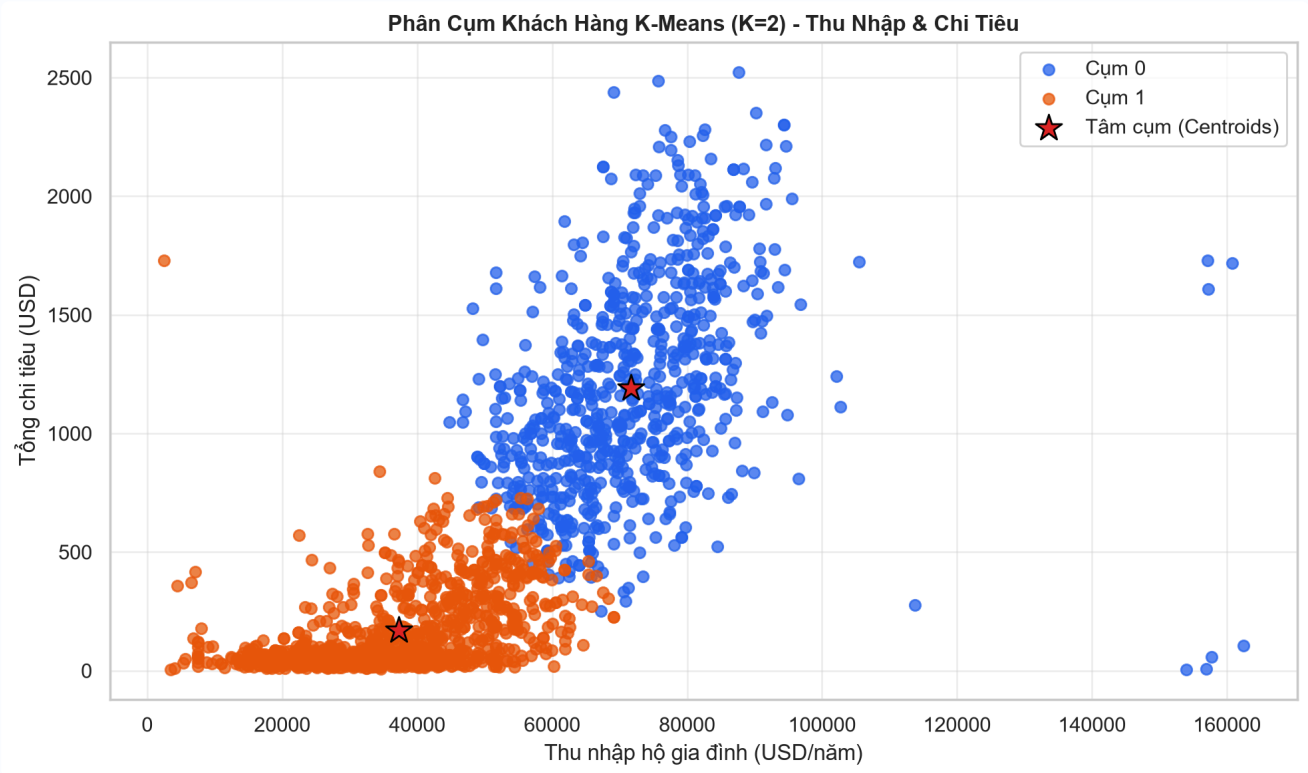
Kết quả áp mô hình đã học lên tập Holdout Test độc lập ( `holdout_test_evaluation.csv` )

| Mô hình (áp tâm cụm Train sang Test) | Số cụm | Silhouette Test | Silhouette âm (%) | Davies–Bouldin | Calinski–Harabasz | Dunn Index | Độ cân bằng cụm |
|---|---|---|---|---|---|---|---|
| **K-Means Test Holdout (K=2)** | **2** | **0,4453** | **2,01%** | **0,8977** | **455,16** | **0,0291** | **0,7231** |
| K-Medoids Test Holdout (K=2) | 2 | 0,4418 | 2,68% | 0,9023 | 451,75 | 0,0316 | 0,7708 |

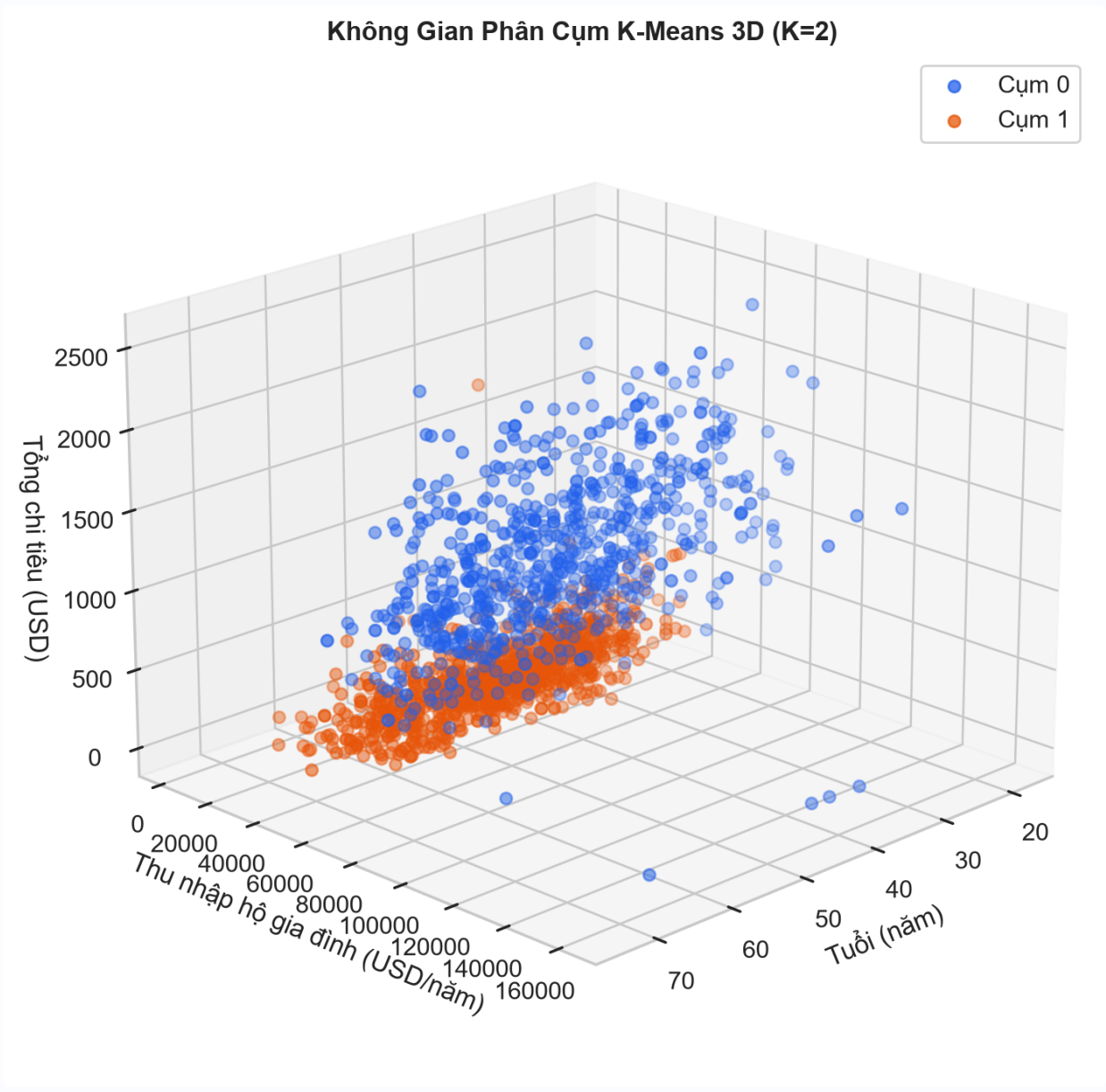
Trên 448 khách hàng mới của tập Test, K-Means gán `188` người vào Cụm 0 ( `41,96%` ) và `260` người vào Cụm 1 ( `58,04%` ), rất sát tỷ lệ `42,67% / 57,33%` ở tập Train ( `test_predictions.csv` ).

> **Việc tính Silhouette trên Holdout Test có dùng để khẳng định "mô hình không bị overfitting" như học có giám sát không?**
>
> Không nên diễn giải như vậy. Do phân cụm không có nhãn chuẩn, các chỉ số nội tại trên Holdout Test chỉ giúp kiểm tra xem **ranh giới và tâm cụm đã học có giữ được cấu trúc phân tách nhất quán** khi gặp dữ liệu mới cùng phân phối hay không, chứ không phải phép đo sai số khái quát hóa (generalization error) như bài toán phân loại/hồi quy có nhãn.

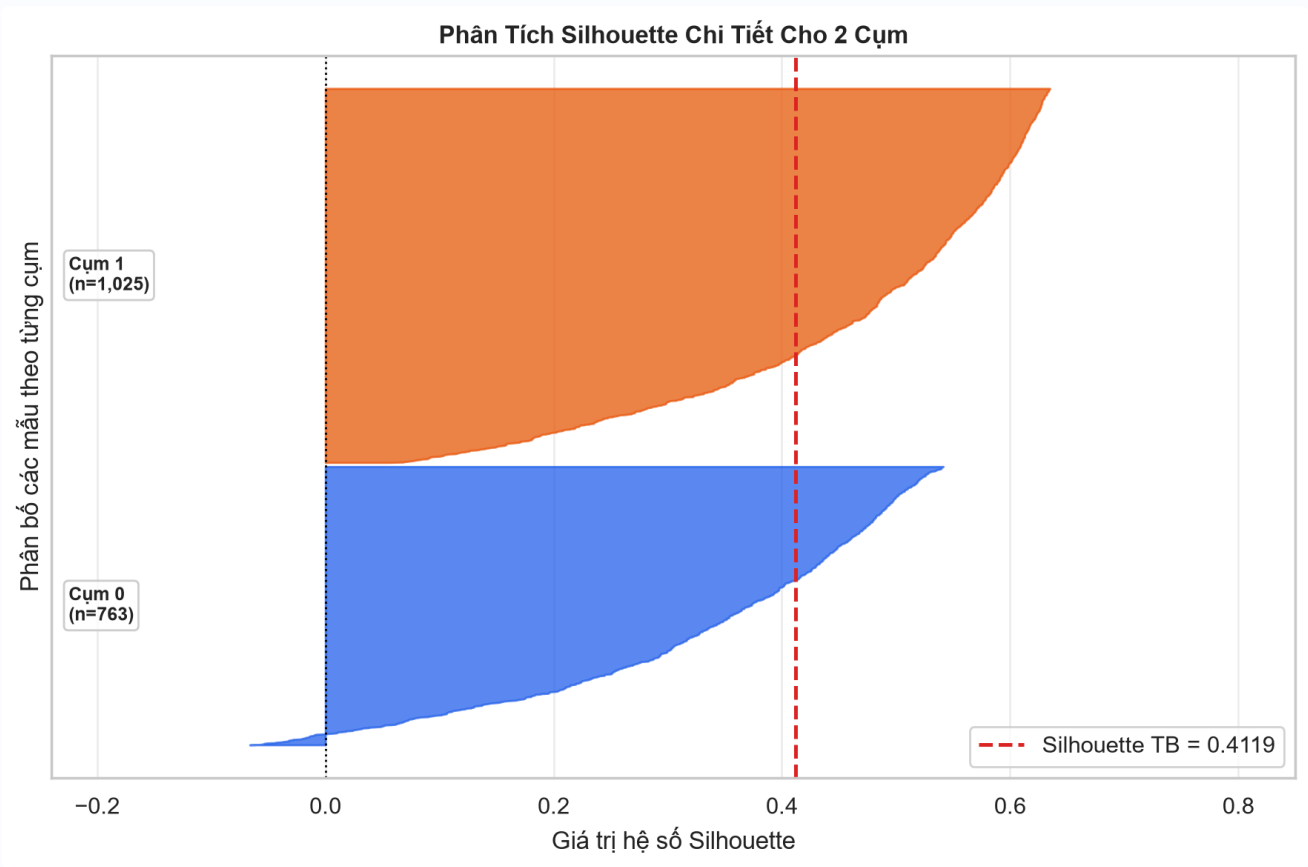
### 8.3. Trực quan hóa mô hình K-Means được chọn (K = 2)




Hình 8: Phân bố 2 cụm K-Means kèm tâm cụm quy đổi về thang đo gốc (Income vs Total_Spending)




Hình 9: Không gian phân cụm 3D (Age x Income x Total_Spending) của mô hình K-Means




Hình 10: Silhouette Knife Plot chi tiết từng cụm của K-Means (K = 2) — đường nét đứt thể hiện mức trung bình 0,4119

## 9. Chân dung phân khúc khách hàng và phân tích điểm biên

### 9.1. Hồ sơ thống kê 2 cụm khách hàng từ tập Train ( `cluster_profiles.csv` )

Thống kê đặc điểm nhân khẩu học và hành vi chi tiêu theo cụm (K-Means, K = 2)

| Cụm | Số lượng (%) | Tuổi TB (Trung vị) | Thu nhập TB (Trung vị) | Chi tiêu TB (Trung vị) | Học vấn phổ biến | Hôn nhân phổ biến | Danh mục chi chính |
|---|---|---|---|---|---|---|---|
| **Cụm 0** VIP / Giá trị cao | 763 (42,67%) | 47,64 (48,0) | 71.704,57( 70.647,0) | 1.191,48( 1.150,0) | Graduation (52,9%) | Married (37,4%) | MntWines (TB: 605,6) |
| **Cụm 1** Tiết kiệm / Nhạy cảm giá | 1.025 (57,33%) | 43,43 (42,0) | 37.289,47( 37.401,0) | 169,48(85,0) | Graduation (49,3%) | Married (40,2%) | MntWines (TB: 81,7) |

- **Cụm 0 — Khách hàng VIP / Giá trị cao (42,67%):** Độ tuổi nhỉnh hơn (trung bình ~48 tuổi), thu nhập trung bình hơn `$71.700` và tổng chi tiêu gần gấp 7 lần Cụm 1 (trung vị `$1.150` so với `$85` ). Chiếm phần lớn doanh thu và chi mạnh cho rượu vang ( `MntWines` trung bình `605,6` ). *Gợi ý:* chăm sóc hội viên thân thiết, ưu đãi cá nhân hóa theo các nhóm sản phẩm cao cấp.

- **Cụm 1 — Khách hàng Tiết kiệm / Nhạy cảm giá (57,33%):** Chiếm đa số về số lượng người dùng nhưng chi tiêu trung vị chỉ `$85`, thu nhập trung bình khoảng `$37.289` và độ tuổi trẻ hơn (trung vị 42 tuổi). *Gợi ý:* combo sản phẩm thiết yếu giá tốt, tích điểm đổi quà và khuyến mãi theo dịp.

### 9.2. Phân tích nhóm điểm biên có Silhouette âm (Boundary Analysis)

Do phân cụm không có nhãn đúng, thay cho phân tích lỗi sai nhãn, bài lab kiểm tra trực tiếp **30 mẫu (1,68%)** trên tập Train có hệ số Silhouette âm ($s_i < 0$, nghĩa là điểm nằm gần ranh giới cụm láng giềng hơn cụm được gán). Kiểm tra thực nghiệm trên mô hình `final_kmeans_pipeline.pkl` cho thấy toàn bộ 30 điểm này đều thuộc **Cụm 0** (Cụm 1 có 0 điểm âm):

5 khách hàng có Silhouette âm thấp nhất trên tập Train (trích kiểm tra trực tiếp từ mô hình đã lưu)

| ID khách hàng | Tuổi (Age) | Thu nhập (Income) | Tổng chi (Total_Spending) | Cụm được gán | Silhouette mẫu | Lý giải vùng giao thoa |
|---|---|---|---|---|---|---|
| 8970 | 42 | $62.010 | $530 | 0 | -0,0663 | Thu nhập tiệm cận Cụm 0 nhưng chi tiêu mức trung bình thấp gần tâm Cụm 1 |
| 1343 | 46 | $58.554 | $586 | 0 | -0,0585 | Nằm đúng dải chuyển tiếp thu nhập ~58k và chi tiêu ~586 |
| 3594 | 41 | $54.108 | $747 | 0 | -0,0554 | Tuổi gần Cụm 1, thu nhập và chi tiêu ở giữa hai tâm cụm |
| 10704 | 45 | $54.803 | $693 | 0 | -0,0541 | Tất cả 3 chiều đều nằm sát mặt phẳng trung trực của 2 tâm cụm |
| 10476 | 56 | $67.267 | $251 | 0 | -0,0537 | Thu nhập khá cao (kéo về Cụm 0) nhưng chi tiêu thực tế rất ít ($251) |

Đây là bằng chứng trực quan cho thấy trong thực tế kinh doanh vẫn tồn tại một vùng chuyển tiếp (khách hàng thu nhập khá nhưng chi tiêu dè dặt); phép phân cụm cứng K-Means buộc mỗi điểm phải thuộc hẳn một phía của siêu phẳng ranh giới thay vì phản ánh độ mờ vùng biên.

## 10. Đóng gói mô hình và tiện ích dự đoán thử nghiệm (CLI)

Để minh họa bước sử dụng lại mô hình đã huấn luyện trong bài lab, `src/models/persistence.py` đóng gói `final_kmeans_pipeline.pkl` gồm: đối tượng `KMeans` ($K = 2$), đối tượng `FeatureScaler` đã fit trên Train, cùng `metadata` (`feature_cols`, `scaler_method`, `n_clusters`, `reference_year=2014`, `feature_medians` của tập Train để điền khuyết `Income` nếu dữ liệu mới bị thiếu, và danh sách `personas`).

Lớp `CustomerSegmentPredictor` trong `src/deployment/predictor.py` hỗ trợ suy luận một khách hàng mới hoặc một tệp dữ liệu hàng loạt qua dòng lệnh; đồng thời kiểm tra nghiêm ngặt miền dữ liệu (chặn giá trị không hữu hạn, tuổi ngoài `(0, 120]`, thu nhập/tổng chi/khoản chi thành phần `Mnt*` âm kèm index dòng lỗi) và dùng trung vị Train trong `feature_medians` để điền giá trị khuyết:

```
.\.venv\Scripts\python.exe -m src.deployment.predictor --age 35 --income 75000 --spending 1400

# Kết quả thực tế in ra:
# Cụm dự đoán: 0 - Khách hàng VIP / Giá trị cao (Trung niên/Cao tuổi)
```

```
# Khoảng cách tới tâm cụm: 1.1328
# Chiến lược gợi ý: Ưu tiên chăm sóc hội viên VIP, truyền thông các danh mục chi tiêu chủ lực và cá nhân hóa ưu đãi duy trì mức chi tiêu cao.
```

> **Giới hạn triển khai:** Module đóng gói và CLI này chỉ dùng ở phạm vi mô phỏng suy luận và kiểm thử trong bài thực hành, không phải dịch vụ API web (FastAPI/Flask) hay hệ thống sản xuất có giám sát độ trôi dữ liệu (data drift).

## 11. Kiểm thử tự động và hướng dẫn chạy lại

### 11.1. Kết quả kiểm thử đơn vị ( `tests/` )

Dự án có bộ kiểm thử tự động gồm **38 test cases** chia đều theo 5 module ( `test_data.py` , `test_features.py` , `test_models.py` , `test_evaluation.py` , `test_deployment.py` ), kiểm tra tính đúng đắn của công thức tính tay (IQR, Dunn Index, K-Medoids medoid là điểm thật, chống data leakage khi fit scaler, chặn NaN/Inf/ `Mnt*` âm, bảo vệ các trường hợp DBSCAN chỉ tạo 1 cụm không gán giả 0, và kiểm tra vòng đời lưu/nạp `.pkl` + CLI). Khi chạy:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Kết quả xác nhận **38/38 tests đạt** `OK` . Lưu ý rằng việc vượt qua 38 unit tests chứng minh mã nguồn chạy đúng các trường hợp đã kiểm tra trong bài lab chứ không thay thế đánh giá chất lượng mô hình trên dữ liệu ngoài tập mẫu.

### 11.2. Môi trường và cách chạy lại toàn bộ bài lab

Phiên bản môi trường đã chạy kiểm chứng trong `.venv` : **Python 3.12.10**, `NumPy 2.5.3` , `SciPy 1.18.1` , `Pandas 3.0.6` , `Scikit-learn 1.9.1` , `Matplotlib 3.11.2` , `Seaborn 0.13.2` , `Joblib 1.6.0` .

```
# 1. Kích hoạt môi trường ảo & cài đặt thư viện (nếu chưa có)
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt

# 2. Chạy toàn bộ pipeline (lưu kết quả vào data/processed/ và results/)
.\.venv\Scripts\python.exe main.py

# 3. Chạy bộ kiểm thử đơn vị (38 tests)
.\.venv\Scripts\python.exe -m unittest discover -s tests -v

# 4. Hoặc mở & chạy tuần tự notebooks/01_eda.ipynb -> 04_evaluation_and_profiling.ipynb
# (đầu ra được tách riêng tại notebooks/results/ để không ghi đè main.py)
```

## 12. Giới hạn của bài lab và hướng mở rộng tiếp theo

1. **Giới hạn dữ liệu lịch sử và thiếu biến giới tính:** Tập `marketing_campaign.csv` thu thập giai đoạn `2012–2014` nên mức thu nhập và chi tiêu phản ánh bối cảnh cũ; ngoài ra không có biến `Gender` như mô tả khái quát của đề

gốc.

2. **Gộp biến làm mất chi tiết hành vi:** Việc gộp 6 cột `Mnt*` thành 1 chiều `Total_Spending` giúp bám sát không gian 3 chiều của đề và vẽ được biểu đồ 2D/3D trực quan, nhưng làm mất thông tin về cơ cấu giỏ hàng hay kênh mua sắm ( `NumWebPurchases` , `NumStorePurchases` , `Recency` ).
3. **Độ lệch phân phối của chi tiêu:** Với `skew = 0,8565` , một số ít khách hàng thu nhập cao nhưng ít chi tiêu vẫn gây hiện tượng 30 điểm biên Silhouette âm ( `1,68%` ). Bài lab hiện chưa thử phép biến đổi `log1p(Total_Spending)` hay phân cụm mềm (Gaussian Mixture Model / Fuzzy C-Means).
4. **Mật độ liên tục và cấu trúc 2 cụm:** Do `Income` và `Total_Spending` tương quan thuận khá mượt mà không tạo thành nhiều "đảo" rời rạc, cả các thuật toán phân hoạch lẫn phân cấp đều dừng ở $K = 2$. Muốn tách sâu hơn 4–5 phân khúc marketing chi tiết, cần bổ sung thêm các đặc trưng RFM (Recency–Frequency–Monetary) hoặc sở thích nhóm hàng.

## 13. Hồ sơ nguồn đối chiếu và mức độ tin cậy kết luận

Theo nguyên tắc minh bạch chứng cứ, toàn bộ con số trong báo cáo này có thể kiểm tra chéo trực tiếp tại các tệp nguồn trong repository:

Bảng truy nguyên nguồn số liệu và đánh giá mức độ tin cậy của từng kết luận

| Kết luận / Nhóm số liệu | Tệp nguồn trực tiếp trong repo | Mức độ kiểm chứng | Phạm vi áp dụng hợp lệ |
|---|---|---|---|
| Kiểm toán dữ liệu thô, loại 4 dị biệt, điền trung vị 51.650,50 | `src/data/loader.py` , `preprocessing.py` , `raw_data_audit.csv` , `train_iqr_audit.csv` | Cao (kiểm chứng bởi `tests/test_data.py` ) | Trên tập `marketing_campaign.csv` với mốc năm 2014 |
| Lựa chọn `StandardScaler` thay cho `MinMaxScaler` và `RobustScaler` | `src/features/scaling.py` , `scaler_comparison.csv` , `scaler_verification.csv` , `scaler_clustering_comparison.csv` | Cao (có đối chiếu định lượng cả 3 scaler trên Train & Test) | Trên không gian 3 đặc trưng thô chưa biến đổi log |
| Cấu trúc tối ưu tại $K = 2$ ( `K-Means` dẫn đầu với Silhouette `0,4119` ) | `kmeans_k_search.csv` , `hierarchical_k_search.csv` , `kmedoids_k_search.csv` , `model_comparison.csv` | Cao (đồng thuận độc lập giữa cả 3 thuật toán khoảng cách) | Chỉ khẳng định trên 3 chiều `[Age, Income, Total_Spending]` ; chưa đại diện nếu thêm biến RFM |
| Hiệu ứng chaining ở `complete` / `average` & mật độ liên tục ở DBSCAN | `hierarchical_cophenetic.csv` , `dbscan_grid_search.csv` | Trung bình – Cao (đã quét 3 linkage và 24 cấu hình DBSCAN) | Phạm vi lưới `eps` ∈ `[0.3..0.8]` trên không gian `StandardScaler` |
| Phân bố cụm ổn định trên Holdout Test (448 mẫu, Silhouette `0,4453` ) | `holdout_test_evaluation.csv` , `test_predictions.csv` | Trung bình (chứng minh ổn định tâm cụm trên dữ liệu chia 80/20 cùng nguồn) | Không dùng thay thước đo generalization của học có giám sát |
| Hồ sơ 2 phân khúc & 30 điểm biên Silhouette âm | `cluster_profiles.csv` , `final_kmeans_pipeline.pkl` | Cao về mặt thống kê mô tả; gợi ý tiếp thị mang tính tham khảo định tính | Phục vụ minh họa diễn giải kết quả trong bài lab |

Các tài liệu bổ sung khác trong dự án gồm: docs/requirements_mapping.md (bảng đối chiếu yêu cầu đề bài) và README.md.

Báo cáo thực hành môn Học máy – Practice Exercise 3: Phân cụm khách hàng (Customer Clustering). Số liệu đối chiếu trực tiếp từ mã nguồn và thư mục `results/` trong bài lab.